

Contents

1	GharBazar - Property Rental Marketplace	2
1.1	Project Overview	2
1.2	Tech Stack	2
1.2.1	Frontend (Web Application)	2
1.2.2	Backend (API Server)	3
1.3	Architecture	4
1.3.1	System Architecture	4
1.4	Project Structure	5
1.4.1	Frontend Structure	5
1.4.2	Backend Structure	6
1.5	Working Flow	7
1.5.1	1. User Authentication Flow	7
1.5.2	2. Property Listing Flow	8
1.5.3	3. Property Search Flow	9
1.5.4	4. Real-time Chat Flow	10
1.5.5	5. Favorites Flow	11
1.5.6	6. Analytics Flow	12
1.6	Security Features	13
1.6.1	Authentication & Authorization	13
1.6.2	API Security	13
1.6.3	WebSocket Security	13
1.6.4	Data Protection	13
1.7	Key Features	13
1.7.1	For Tenants	13
1.7.2	For Owners	13
1.7.3	Admin Features	14
1.8	Database Schema	14
1.8.1	Core Models	14
1.9	Environment Configuration	15
1.9.1	Frontend (.env.local)	15
1.9.2	Backend (.env)	15
1.10	Deployment	16
1.10.1	Frontend Deployment (Vercel)	16
1.10.2	Backend Deployment (Railway/Heroku)	16
1.10.3	Required Services	16
1.11	Performance Optimizations	16
1.11.1	Frontend	16
1.11.2	Backend	16
1.12	Testing	17
1.12.1	Frontend Tests	17
1.12.2	Backend Tests	17
1.13	API Documentation	17
1.14	Contributing	17
1.15	License	17

1.16	Team	17
1.17	Support	18

1 GharBazar - Property Rental Marketplace

1.1 Project Overview

GharBazar is a modern, full-stack property rental marketplace platform that connects property owners with tenants directly, eliminating the need for brokers. The platform features real-time chat, property management, favorites, analytics, and a mobile-first responsive design.

1.2 Tech Stack

1.2.1 Frontend (Web Application)

1.2.1.1 Core Framework

- **Next.js 15.5.12** - React framework with App Router
- **React 19** - UI library with latest features
- **TypeScript** - Type-safe development

1.2.1.2 Styling & UI

- **Tailwind CSS 3.4.1** - Utility-first CSS framework
- **Lucide React** - Modern icon library
- **Custom CSS Animations** - Smooth transitions and effects

1.2.1.3 State Management

- **Zustand** - Lightweight state management
- **TanStack Query (React Query)** - Server state management
- **Persist Middleware** - State persistence

1.2.1.4 Real-time Communication

- **Native WebSocket** - Real-time messaging
- **Django Channels** - WebSocket backend integration
- **React Hot Toast** - Toast notifications

1.2.1.5 Authentication

- **JWT (JSON Web Tokens)** - Secure authentication
- **Firebase Auth** - Google OAuth integration
- **HTTP-only Cookies** - Secure token storage

1.2.1.6 HTTP Client

- **Axios** - Promise-based HTTP client
- **Interceptors** - Request/response handling
- **Error handling** - Centralized error management

1.2.1.7 Development Tools

- **ESLint** - Code linting
 - **TypeScript** - Static type checking
 - **Hot Module Replacement** - Fast refresh
-

1.2.2 Backend (API Server)

1.2.2.1 Core Framework

- **Django 5.1.4** - Python web framework
- **Django REST Framework 3.15.2** - RESTful API toolkit
- **Python 3.11+** - Programming language

1.2.2.2 Database

- **PostgreSQL** - Primary relational database
- **psycopg2-binary 2.9.10** - PostgreSQL adapter

1.2.2.3 Real-time Features

- **Django Channels 4.1.0** - WebSocket support
- **Channels Redis 4.2.0** - Channel layer backend
- **Redis** - Message broker and caching

1.2.2.4 Authentication & Security

- **django-rest-framework-simplejwt 5.4.0** - JWT authentication
- **Firebase Admin SDK** - Google OAuth verification
- **CORS Headers** - Cross-origin resource sharing
- **django-cors-headers 4.6.0** - CORS middleware

1.2.2.5 File Storage

- **Pillow 11.0.0** - Image processing
- **Django Storage** - File upload handling

1.2.2.6 Task Queue

- **Celery 5.4.0** - Asynchronous task queue
- **Redis** - Celery broker

1.2.2.7 API Documentation

- **drf-spectacular 0.28.0** - OpenAPI 3.0 schema generation
- **Swagger UI** - Interactive API documentation

1.2.2.8 Testing

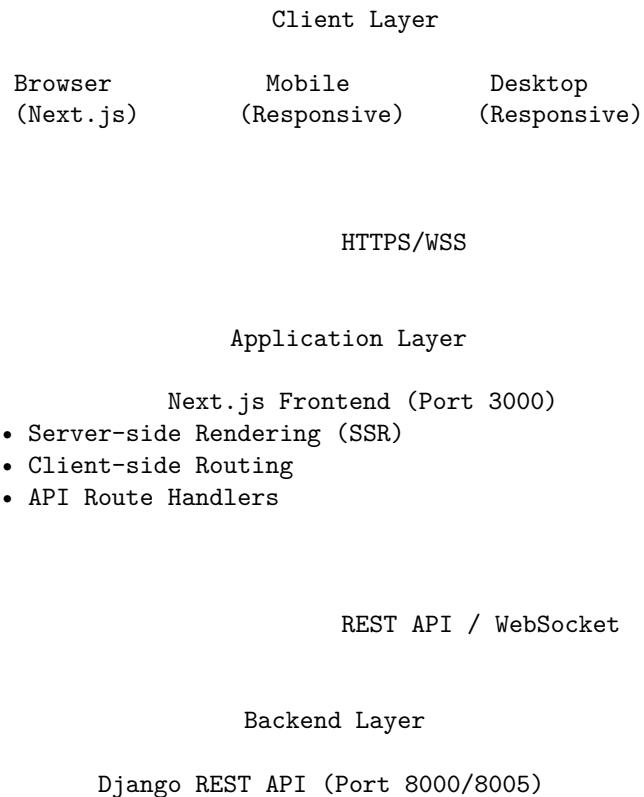
- **pytest** - Testing framework
- **pytest-django** - Django testing utilities
- **Coverage** - Code coverage reporting

1.2.2.9 Development Tools

- **python-dotenv** - Environment variable management
 - **django-extensions** - Development utilities
-

1.3 Architecture

1.3.1 System Architecture



- RESTful Endpoints
- JWT Authentication
- Business Logic
- WebSocket (Django Channels)

PostgreSQL Database	Redis Cache & Channels	Celery Task Queue
------------------------	------------------------------	----------------------

1.4 Project Structure

1.4.1 Frontend Structure

```

Apps/webapp/
  app/
    account/
    auth/
    chats/
    dashboard/
      properties/
        add/
        page.tsx
    profile/
    properties/
      [id]/
        page.tsx
    signup/
    owner-signup/
    layout.tsx
    page.tsx
    globals.css
  src/
    components/
    layout/
    property/
    providers/
    hooks/
    # Next.js App Router
    # Account hub page
    # Authentication pages
    # Real-time chat interface
    # User dashboard
    # Property management
    # Add property form
    # My properties list
    # User profile page
    # Property listings
    # Property detail page
    # Property search
    # Tenant signup
    # Owner signup
    # Root layout
    # Homepage
    # Global styles
    # Reusable components
    # Layout components
    # Property components
    # Context providers
    # Custom React hooks

```

modules/	# Feature modules
auth/	# Authentication
communication/	# Chat system
contacts/	# Contact leads
favorites/	# Favorites
properties/	# Properties
services/	# API services
http.ts	# HTTP client
websocket.ts	# WebSocket service
store/	# Zustand stores
types/	# TypeScript types
lib/	# Utility functions
public/	# Static assets

1.4.2 Backend Structure

Server/	
apps/	# Django apps
analytics/	# Analytics & tracking
communication/	# Chat & messaging
consumers.py	# WebSocket consumers
routing.py	# WebSocket routing
models.py	# Conversation & Message
serializers.py	# API serializers
services.py	# Business logic
views.py	# API views
contacts/	# Contact leads
favorites/	# Favorites system
notifications/	# Notifications
properties/	# Property management
models.py	# Property model
serializers.py	# API serializers
services.py	# Business logic
filters.py	# Query filters
views.py	# API views
users/	# User management
core/	# Core utilities
models.py	# Base models
permissions.py	# Custom permissions
throttles.py	# Rate limiting
exceptions.py	# Custom exceptions
server/	# Django settings
settings.py	# Configuration
urls.py	# URL routing
asgi.py	# ASGI config
wsgi.py	# WSGI config

manage.py

Django CLI

1.5 Working Flow

1.5.1 1. User Authentication Flow

Landing
Page

Sign Up / Firebase
Sign In OAuth

(Google Token)

Backend
Validates
Token

JWT Token
Generated

Stored
in Zustand
& Cookie

Dashboard
Redirect

1.5.2 2. Property Listing Flow

Owner Journey:

Owner
Dashboard

Add Property
Form
(4 Steps)

Step 1: Property Type & Details
Step 2: Location (Create/Select)
Step 3: Pricing & Amenities
Step 4: Review & Submit

Backend
Creates
Property
(DRAFT)

Submit
for
Approval
(PENDING)

Admin
Activates
Property
(ACTIVE)

Visible
to
Tenants

1.5.3 3. Property Search Flow

Tenant Journey:

Browse
Properties

Search
Filters
Applied

City
Property Type
Min/Max Rent
Search Query

Backend
Filters
& Searches

Property
List
Displayed

Select
Property

Property
Detail
Page

Add to Favorites
Chat with Owner

1.5.4 4. Real-time Chat Flow

Tenant
Clicks
"Chat with
Owner"

Create
Conversation
(REST API)

Redirect
to
Chat Page

WebSocket
Connection
Established

Send
Message

Django
Channels

Redis
Broadcasts

Receive
Message
(Real-time)

1.5.5 5. Favorites Flow

Tenant
Clicks
Heart
Icon

POST /api/
favorites/

Backend
Creates
Favorite

Update
UI
(Filled
Heart)

Visible
in

Dashboard
Wishlist

1.5.6 6. Analytics Flow

User
Action

Property View
Favorite Added
Contact Created

Backend
Increments
Counters

Property
Stats
Updated

total_views++
total_favorites++
total_contacts++

Owner
Dashboard
Shows
Stats

1.6 Security Features

1.6.1 Authentication & Authorization

- JWT token-based authentication
- HTTP-only cookies for token storage
- Role-based access control (TENANT/OWNER)
- Protected routes with auth guards
- Token refresh mechanism

1.6.2 API Security

- CORS configuration
- Rate limiting (throttling)
- Input validation
- SQL injection prevention (ORM)
- XSS protection

1.6.3 WebSocket Security

- JWT authentication on connection
- User verification middleware
- Conversation access control
- Message validation

1.6.4 Data Protection

- Password hashing (Django default)
 - Sensitive data encryption
 - Environment variable management
 - Secure file uploads
-

1.7 Key Features

1.7.1 For Tenants

- Browse properties with advanced filters
- Search by location, type, rent range
- Save favorite properties
- Real-time chat with owners
- View property details & images
- Track inquiries and responses
- Mobile-first responsive design

1.7.2 For Owners

- List properties (4-step form)

- Manage property listings
- View analytics (views, favorites, contacts)
- Real-time chat with tenants
- Mark properties as rented
- Track property performance
- Dashboard with insights

1.7.3 Admin Features

- Approve/reject property listings
 - Manage users
 - View system analytics
 - Moderate content
-

1.8 Database Schema

1.8.1 Core Models

1.8.1.1 User

- id (UUID)
- email (unique)
- name
- phone
- role (TENANT/OWNER)
- location (FK)
- created_at, updated_at

1.8.1.2 Property

- id (UUID)
- owner (FK → User)
- location (FK → Location)
- title
- description
- property_type
- rent
- furnishing
- status (DRAFT/PENDING/ACTIVE/RENTED)
- total_views, total_favorites, total_contacts
- created_at, updated_at

1.8.1.3 Conversation

- id (UUID)
- property (FK → Property)

- tenant (FK → User)
- owner (FK → User)
- status (ACTIVE/ARCHIVED)
- message_count
- owner_unread_count, tenant_unread_count
- last_message_at

1.8.1.4 Message

- id (UUID)
- conversation (FK → Conversation)
- sender (FK → User)
- message_type (TEXT/IMAGE/SYSTEM)
- content
- is_read
- created_at

1.8.1.5 Favorite

- id (UUID)
- user (FK → User)
- property (FK → Property)
- created_at

1.9 Environment Configuration

1.9.1 Frontend (.env.local)

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
NEXT_PUBLIC_FIREBASE_API_KEY=your_firebase_key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your_domain
NEXT_PUBLIC_FIREBASE_PROJECT_ID=your_project_id
```

1.9.2 Backend (.env)

```
DEBUG=True
SECRET_KEY=your_secret_key
DATABASE_URL=postgresql://user:pass@localhost:5432/gharbazar
REDIS_URL=redis://localhost:6379/0
FIREBASE_CREDENTIALS_PATH=config/firebase-admin.json
ALLOWED_HOSTS=localhost,127.0.0.1
CORS_ALLOWED_ORIGINS=http://localhost:3000
```

1.10 Deployment

1.10.1 Frontend Deployment (Vercel)

Build

```
npm run build
```

Deploy

```
vercel --prod
```

1.10.2 Backend Deployment (Railway/Heroku)

Collect static files

```
python manage.py collectstatic
```

Run migrations

```
python manage.py migrate
```

Start server

```
daphne -b 0.0.0.0 -p $PORT server.asgi:application
```

1.10.3 Required Services

- PostgreSQL database
 - Redis server
 - Celery worker
 - File storage (AWS S3/Cloudinary)
-

1.11 Performance Optimizations

1.11.1 Frontend

- Server-side rendering (SSR)
- Image optimization
- Code splitting
- Lazy loading
- React Query caching
- Debounced search

1.11.2 Backend

- Database indexing
- Query optimization
- Redis caching
- Connection pooling
- Pagination
- Async tasks (Celery)

1.12 Testing

1.12.1 Frontend Tests

`npm test`

1.12.2 Backend Tests

`python manage.py test`
`pytest`
`coverage run -m pytest`

1.13 API Documentation

Interactive API documentation available at: - **Swagger UI**: <http://localhost:8000/api/schema/swagger-ui/>
- **ReDoc**: <http://localhost:8000/api/schema/redoc/> - **OpenAPI**
Schema: <http://localhost:8000/api/schema/>

1.14 Contributing

1. Fork the repository
 2. Create feature branch (`git checkout -b feature/amazing-feature`)
 3. Commit changes (`git commit -m 'Add amazing feature'`)
 4. Push to branch (`git push origin feature/amazing-feature`)
 5. Open Pull Request
-

1.15 License

This project is proprietary and confidential.

1.16 Team

- **Developer**: Durgesh
 - **Project**: GharBazar Property Rental Marketplace
 - **Year**: 2025
-

1.17 Support

For issues or questions: - Check documentation - Review API docs - Check server logs - Review browser console

Last Updated: January 2025

Version: 1.0.0

Status: Production Ready